Ejercicio de Aplicación 🤓

L8: Algoritmos de boosting

Es hora de que pongas en práctica todo lo aprendido. 🤓

Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que a continuación encontrarás diferentes **desafíos** que podrás resolver de forma independiente y a tu ritmo.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😉

¡Manos a la obra!

1. **Desafío 🎯**

   Tu desafío es construir un modelo de clasificación binaria utilizando el algoritmo ***Gradient Boosting****, aplicando buenas prácticas de validación, análisis de resultados y ajuste de hiperparámetros.*

   Pasos:

   - ➔ *Cargar un dataset de clasificación binaria (puede ser load_breast_cancer de Scikit-learn).*
   - ➔ *Separar los datos en entrenamiento y prueba.*
   - ➔ *Entrenar un modelo GradientBoostingClassifier con hiperparámetros por defecto.*
   - ➔ *Evaluar el desempeño con métricas como accuracy, precision, recall y F1-score.*
   - ➔ *Analizar la importancia de las variables (feature_importances_). Mejorar el modelo mediante el ajuste de hiperparámetros con GridSearchCV.*

2. **¿Dónde se lleva a cabo? 🧑‍💻**

   Visual Studio Code

3. **Tiempo de dedicación ⏳**

   2 Horas.

4. **Recursos🛠️**

   ➔ https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html

   ➔ https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html

5. **Plus ➕**

   Comparar el rendimiento de Gradient Boosting con otro modelo ensemble, como Random Forest.

6. **⚠️Condición**

   Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Ejercicio de Aplicación 🤓

L8: Algoritmos de boosting

Resolución del desafío 👇

```python
# 1. Importar librerías
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 2. Cargar dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# 3. Separar entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Entrenar modelo básico
model = GradientBoostingClassifier(random_state=42)
model.fit(X_train, y_train)

# 5. Evaluación inicial
y_pred = model.predict(X_test)
print("Resultados iniciales:\n")
print(classification_report(y_test, y_pred, target_names=data.target_names))

# 6. Importancia de características
importances = model.feature_importances_
feat_imp = pd.Series(importances, index=data.feature_names).sort_values(ascending=False)
feat_imp.plot(kind='bar', figsize=(10,5), title="Importancia de las variables")
plt.tight_layout()
plt.show()
```

```python
# 7. Búsqueda de hiperparámetros
param_grid = {
    'n_estimators': [50, 100],
    'learning_rate': [0.1, 0.01],
    'max_depth': [3, 4]
}
grid = GridSearchCV(GradientBoostingClassifier(random_state=42), param_grid, cv=3)
grid.fit(X_train, y_train)

print("Mejores parámetros:", grid.best_params_)

# 8. Evaluación del mejor modelo
y_pred_best = grid.predict(X_test)
print("\nResultados tras optimización:")
print(classification_report(y_test, y_pred_best, target_names=data.target_names))
```